

Hands-on 1: Spring Data JPA

Introduction

Most enterprise software relies on relational databases to store and retrieve application data. Writing SQL statements for every database interaction can become repetitive, increase development effort, and make applications more difficult to maintain. Spring Data JPA simplifies this process by integrating the Java Persistence API (JPA) with the Spring Framework. Instead of writing SQL for routine operations, developers can interact with database records through Java objects and repository interfaces.

Persistence in Spring Applications

Persistence is the process of storing application data permanently so that it remains available even after the application stops running. In a Spring Boot application, the persistence layer manages communication with the database while allowing the business logic to remain independent of database-specific code.

Spring Data JPA serves as the abstraction layer between the application and the database. Internally, it relies on a JPA implementation—typically Hibernate—to translate Java objects into SQL operations and convert query results back into Java objects.

Object-Relational Mapping (ORM)

Object-Relational Mapping (ORM) is a technique that bridges object-oriented programming and relational databases. Instead of manually writing SQL queries for every operation, developers create Java classes that represent database tables. The ORM framework automatically handles the mapping between Java objects and database records.

Mapping Between Java Objects and Database Elements

Java Component Database Equivalent

Class	Table
Object	Row (Record)
Field	Column
Object Reference	Foreign Key

Advantages of ORM

- Reduces the amount of repetitive SQL code.
- Produces cleaner and more maintainable applications.
- Makes it easier to switch between database systems.

- Synchronizes Java objects with database records automatically.
- Simplifies transaction management.

Drawbacks

- Complex queries may require optimization for better performance.
- Beginners may need time to understand ORM concepts.
- Native SQL queries are sometimes necessary for advanced scenarios.

Core Components of Spring Data JPA

Entity Class

An entity is a Java class that corresponds to a table in the database. Each instance of the entity represents one row within that table.

```
@Entity
```

```
@Table(name = "country")
```

```
public class Country {
```

```
    @Id
```

```
    private String code;
```

```
    private String name;
```

```
}
```

Repository Layer

The repository layer removes the need to manually implement Data Access Object (DAO) classes. By extending `JpaRepository`, developers automatically gain methods for common database operations such as inserting, updating, deleting, and retrieving records.

```
public interface CountryRepository
```

```
    extends JpaRepository<Country, String> {
```

```
}
```

Service Layer

The service layer contains the application's business logic and acts as an intermediary between the controller and the repository.

Typical responsibilities include:

- Validating incoming data
- Invoking repository methods
- Managing transactions
- Enforcing business rules

Commonly Used JPA Annotations

Annotation	Purpose
@Entity	Marks a class as a persistent entity.
@Table	Specifies the corresponding database table.
@Id	Identifies the primary key field.
@Column	Maps a field to a table column.
@Repository	Declares a repository component.
@Service	Identifies the service layer.
@Autowired	Injects Spring-managed dependencies.
@Transactional	Ensures operations execute within a transaction.

Repository Methods

JpaRepository includes several predefined methods that eliminate the need for writing SQL for common CRUD operations.

Method	Function
findAll()	Returns every record in the table.
findById()	Retrieves a record using its primary key.
save()	Inserts a new record or updates an existing one.

Method	Function
---------------	-----------------

<code>deleteById()</code>	Deletes a record using its identifier.
---------------------------	--

<code>existsById()</code>	Checks if a record exists.
---------------------------	----------------------------

<code>count()</code>	Returns the total number of stored records.
----------------------	---

These methods are available immediately after extending `JpaRepository`.

Query Generation from Method Names

A notable feature of Spring Data JPA is its ability to generate SQL queries automatically based on repository method names.

Examples include:

- `findByName()`
- `findByCode()`
- `findByNameContaining()`
- `findByNameStartingWith()`
- `findByNameEndingWith()`

Spring analyzes these method names at runtime and generates the appropriate SQL queries automatically.

Database Configuration

Database connection settings are defined in the `application.properties` file. Important configuration parameters include:

- JDBC connection URL
- Database username
- Database password
- Hibernate dialect
- DDL generation strategy
- SQL logging options

Spring Boot reads these properties during startup and automatically establishes the required database connection.

Why Choose Spring Data JPA?

Spring Data JPA accelerates application development by reducing the amount of persistence-related code developers need to write.

Some key benefits include:

- Minimal boilerplate code
- Built-in CRUD functionality
- Seamless integration with Spring Boot
- Automatic transaction handling
- Query creation using method names
- Better readability and maintainability
- Faster development of database-driven applications